What is FPGA Programming?

Last time, I presented in detail what exactly an FPGA is and the advantage of FPGAs over ASICs and microcontrollers. FPGAs are nothing, but reconfigurable logic blocks and interconnects can be programmed by Hardware Description Language like Verilog/ VHDL to perform a specific functionality.

Today, I will present what actually FPGA programming is, how to get started with FPGA programming, and FPGA programming design flow. The difference between FPGA programming and software programming will be also discussed.

1. What is FPGA Programming?

As mentioned in the previous FPGA post, FPGAs are nothing, but reconfigurable logic blocks (logic gates, memory elements, DSP components, etc.) and interconnects. *FPGA programming is actually (re)configuring FPGAs using Hardware Description Language (Verilog/VHDL) to connect these logic blocks and interconnects in a way that it can perform a specific functionality (adders, multipliers, processors, filters, dividers, etc.).*

2. What is the FPGA programming language?

FPGA programming language is commonly called Hardware Description Language because it is actually used to describe or design hardware. The two major Hardware Description Languages are Verilog HDL and VHDL.

Verilog = "Verification" + "Logic", which originally created by P. Moorby, P. Goel, C.-L. Huang, and D. Warmke in 1984 to model gates and perform simulation in a logic simulator. Verilog was acquired by Cadence in 1990 and became IEEE Standard 1364 in 1995. Now, Verilog is commonly used for designing and verification of digital circuits and analog or mixed signal circuits as well.

VHDL is VHSIC (Very High Speed Integrated Circuits) Hardware Description Language which was initially developed by US Defense Department to research and develop very high speed integrated circuits in the 1980s. VHDL became IEEE Standard 1076 in 1987. Now, VHDL is mostly used to model digital and analog mixed signal circuits such as FPGAs and ASICs.

Verilog HDL is similar to C programming language such as case-sensitive, keywords, operators, etc. VHDL is similar to Ada programming language since it was initially built based on Ada. The difference between Verilog and VHDL is presented here. *Although there are similarities in syntax between HDLs (Verilog, VHDL) and software programming languages (C, Ada), FPGA programming is completely different from Software programming.*

3. What is the key difference between FPGA programming and software programming?

FPGA Programming vs Software programming

The explicit difference between FPGA programming and software programming is the way that its instructions are executed. *In term of the execution of instructions, instructions in software programming (C, Ada, etc.) are sequentially executed while Verilog/ VHDL instructions in FPGA programming are mostly parallelly executed (except blocking assignment in Verilog and variable assignment within a process in VHDL). More importantly, FPGA programming or FPGA design is about designing digital logic circuits to define the behavior of FPGAs while software programming is about the execution of a sequence of sequential instructions to perform a specific behavior in software.*

When you start FPGA programming, you should forget the coding behavior in software programming. Many students copy directly the way they program (For Loop, While Loop, etc.) in software programming to FPGA programming in order to design and implement synthesizable digital circuits on FPGA. That's a very bad FPGA coding behavior.

Synthesis tools may take a long long time to synthesize this type of code and even worse, synthesis software keeps synthesizing until overflowing RAM's computers and you have to close the software and re-run. *Therefore, always thinking about the logic circuits to implement the functionality that you want while programming FPGA. That's why in FPGA programming, people usually design basic architectures or logic circuits based on projects' specification at first before starting coding*.

4. How to get started with FPGA programming and what is the FPGA programming design flow?

a. Digital logic design background

To get started with FPGA programming, you need to have a good background in digital logic design in order to get familiar with logic values, logic gates, combinational and sequential logic circuits. This is a good resource for it.

b. Verilog/ VHDL language

Then, you can start studying Verilog/ VHDL for programming FPGAs. Verilog/ VHDL is the hardware description language, so as I mentioned, you need to forget the software coding behavior and start thinking about logic gates and circuits to implement the functionality that you want to run on FPGAs. To learn a programming language, my recommended way is to keep practicing design and coding whenever you have time.

c. Simulation

To practice FPGA programming, simulation is a critical step before synthesis and implementation. Simulation is to verify the functional correctness of the Verilog/ VHDL code that you write. There are many simulators out there, but Modelsim is the most common simulator that I used for functional simulation. One more thing, you need to write a test bench which gives all the possible combination of the input values to verify the design. The test bench coding style may not be too restricted, like the synthesizable coding style so you can use software behavior style (For loop, While loop, etc.) to generate the input patterns for functional simulation.

d. Synthesis and Implementation

After successfully verified the design in functional simulation, you can think about the next step of synthesis and Implementation. The two most popular synthesis and implementation tools for FPGAs are Xilinx ISE/ Vivado Design suite for Xilinx FPGAs and Quartus II for Intel Altera FPGAs. It is also noted that you can simulate your design with these tools; simulators (Xilinx ISIM, ModelSim-Altera) are integrated into Xilinx ISE/ Quartus II. The tool enables synthesizing code and generating the bit stream file for FPGA programming.

e. FPGA Development Boards

After obtained the bitstream file from Xilinx ISE/ Vivado/ Quartus II, we will download the bitstream to FPGA boards for demonstrations. I already recommended very cheap FPGA boards as follows:
Recommended and affordable Xilinx FPGA boards for students
Recommended and affordable Altera FPGA boards for students
You need to buy an FPGA board to practice FPGA programming and see what exactly happening on real circuits with the code you wrote.
Basically, that's what you need to get started with FPGA programming and how FPGA programming is done on FPGA boards. I will post FPGA tutorials soon on how to start the first FPGA project on Xilinx/ Intel Altera FPGA boards.
FPGA/ Verilog/ VHDL projects for your practice: FPGA projects, Verilog projects, VHDL projects
Recommended FPGA projects for students:

1. What is FPGA? How does FPGA work?
2. Basys 3 FPGA OV7670 Camera
3. How to load text file or image into FPGA
4. Image processing on FPGA using Verilog
5. License Plate Recognition on FPGA
6. Alarm Clock on FPGA using Verilog
7. Digital Clock on FPGA using VHDL
8. Simple Verilog code for debouncing buttons on FPGA
9. Traffic Light Controller on FPGA
10. Car Parking System on FPGA in Verilog
11. VHDL code for comparator on FPGA
12. Verilog code for Multiplier on FPGA
13. N-bit Ring Counter in VHDL on FPGA
14. Verilog implementation of Microcontroller on FPGA
15. Verilog Carry Look Ahead Multiplier on FPGA
16. VHDL Matrix Multiplication on FPGA Xilinx
17. Fixed Point Matrix Multiplication on FPGA using Verilog
18. Verilog Divider on FPGA
19. VHDL code for Microcontroller on FPGA
20. VHDL code for FIR Filter on FPGA
21. Verilog code for Digital logic components on FPGA
22. Delay Timer Implementation on FPGA using Verilog
23. Single-Cycle MIPS processor on FPGA using Verilog

24. FIFO Verilog Implementation on FPGA
25. FIFO VHDL Implementation on FPGA
26. Verilog D Flip Flop on FPGA
27. Comparator Design on FPGA using Verilog
28. D Flip Flop on FPGA using VHDL
29. Full Adder Design on FPGA using Verilog
30. Full Adder Design on FPGA using VHDL
31. Counters on FPGA with Verilog Testbench
32. RISC Processor Design on FPGA using Verilog
33. Verilog test bench for inout ports on FPGA
34. PWM Generator on FPGA using VHDL
35. Tic Tac Toe Game on FPGA using Verilog
36. VHDL code for ALU on FPGA
37. Verilog code for ALU on FPGA
38. Counter design on FPGA with VHDL test bench
39. Pipelined MIPS Processor on FPGA in Verilog (Part-1)
40. Pipelined MIPS Processor on FPGA in Verilog (Part-2)
41. Pipelined MIPS Processor on FPGA in Verilog (Part-3)
42. Verilog Decoder on FPGA
43. Verilog Multiplexers on FPGA
44. N-bit Adder Design on FPGA in Verilog
45. VHDL ALU on FPGA using N-bit Verilog Adder
46. VHDL Shifter on FPGA
47. Lookup Table VHDL example code on FPGA
48. Coprocessor VHDL Implementation on FPGA
49. Affordable Xilinx FPGA boards for beginners
50. Affordable Altera FPGA boards for beginners

DMCA PROTECTED

Copyright © 2016-2020 FPGA4student.com All Rights Reserved.